

自分の声で、 歌を選ぶ。

音域・難易度・歌い方から、自分に合う歌と練習ポイントを探せる歌唱支援Webアプリ



UtaFit

2026年 実践のためのWebプログラミング
自由制作レポート

探す - 納得する - 練習する、をひとつの体験にする。

テーマ

自分の安定音域と練習目的から、歌いやすい曲と歌い方を探せる歌唱支援Webアプリ「UtaFit」を制作した。

カラオケで曲を選ぶとき、知っている曲という理由だけで決めると、実際には最高音が高すぎたり、裏声への切り替えやロングトーンが難しかったりする。そこで、曲名だけでなく、音域・声区・難易度・テンポ・歌唱指標から曲を探し、選んだ後の練習までつながるサービスをテーマにした。

01 探す	02 診断する	03 練習する
曲名、特徴、雰囲気に加え、音域やテンポなど12種類の条件で絞り込む。	安定音域、歌唱レベル、好み、練習目標から100点満点の相性を計算する。	歌詞上の歌唱注釈を確認し、1フレーズずつ練習済みにできる。

サービスの前提

収録データは実在曲20曲と、注釈デモ用の架空曲1曲である。実在曲の歌詞は保存せず、音域・BPM・難易度も動作確認用サンプル値であることを画面に明記した。

権利への配慮	実在曲の歌詞を掲載しない。注釈体験は架空曲に限定。
精度への配慮	未検証の値を正確な分析結果として見せず、サンプルであることを明示。

サービスの説明

01	検索	入力中検索と12種類の絞り込み。検索条件はURLに残る。
02	診断	音域35点、難易度20点など合計100点で相性を計算。
03	詳細	音域幅、歌唱指標、難所、練習ポイント、似ている曲を確認。
04	注釈	架空歌詞のどの部分で、どの声や技法を使うかを色と線で表示。
05	練習	前後のフレーズとコツを見ながら1行ずつ進め、進捗を端末に保存。

主な画面

/songs 検索・絞り込み・並び替え・0件表示	/recommend 匿名で使える音域・好み診断	/songs/[id] 歌唱指標・注釈・練習・類似曲
-----------------------------	-----------------------------	-------------------------------

実装済み

21曲 / 7種類の並び替え / 12種類の絞り込み / 1行練習 / 制作プログ

設計の説明

画面部品、データ、計算処理を分離した。検索は lib/song-search.ts、推薦は lib/recommendation.ts、歌詞注釈の区間計算は lib/annotations.ts にまとめ、UIに依存しない純粋関数としてテストできるようにした。



推薦スコアの設計

推薦はブラックボックスのAIではなく、次の内訳を合計する。点数だけでなく、最大4件の推薦理由も返すため、利用者が結果を解釈できる。

音域	35点
難易度	20点
ジャンル	10点
曲調	10点
アーティスト	10点
類似曲	10点
練習目標	5点

自分なりにこだわった点

01

推薦理由を説明できるようにした

単に「おすすめ度90点」と表示せず、点数の内訳と「地声最高音が安定音域内」などの理由を表示した。生成AIが一般化しても、判断材料を利用者へ返す設計は重要だと考えた。

02

著作権とデータ精度を曖昧にしなかった

実在曲の歌詞を無断で掲載せず、歌詞注釈は架空曲に限定した。確認していない音域データを正確な分析のように見せず、サンプル値であることを明示した。

03

重なる歌詞注釈を表現した

同じ文字列に複数技法が重なる場合、開始位置と終了位置を境界に歌詞を分割し、各区間に重なる注釈をまとめた。重複部分は二重線で示した。

04

MVPの範囲を決めた

アカウント、お気に入り、履歴、データベース接続、AI注釈候補は将来機能とし、今回は検索・診断・注釈・練習という中心価値に集中した。

学びになったこと・感想

- 要件を機能一覧のまま受け取らず、利用者が困る場面に置き換える必要がある。
- URLと画面状態を同期すると、再読み込みや共有に強いWebサービスになる。
- 複雑な表示は、先にデータ構造と計算処理を決めると実装しやすい。
- UIと計算を分離すると、テストで境界値や例外を確認できる。
- AIにコードを書かせる場合でも、仕様、権利、データの正しさは人が判断しなければならない。

VALIDATION	自動テスト 19件成功	静的解析 成功	本番ビルド 成功
------------	-------------	---------	----------

感想

最初は曲検索機能を中心に考えていたが、設計を進めるうちに「曲を見つけた後、どう練習するか」までつながらないとサービスとして弱いと感じた。そのため、歌詞注釈と1行練習を追加し、探す・納得する・練習するという流れに整理した。

また、生成AIを使うと実装速度は上がるが、候補が多く出る分、何を採用し何を削るかを言語化する必要があった。AIの出力を完成品と考えず、設計案を広げる相手として使い、最後はテストと画面で確かめることが大切だと感じた。

プログラミングは画面を作る作業だけでなく、
信頼できる情報の範囲と、今回は作らないものを決める作業でもある。

AIの活用方法

AIには、要件整理、データ型と画面構成の案出し、実装のたたき台、テスト観点の洗い出し、文章の推敲を手伝ってもらった。一方で、テーマ、優先順位、MVPの境界、著作権への配慮、推薦を説明可能にする方針は人が判断する項目として扱った。

AIに任せしたこと	人が判断したこと	確認したこと
選択肢の展開 / コードの下書き / テスト観点 / 文書の推敲	テーマ / 優先順位 / 権利 / データの扱い / MVPの境界	ソースコード / 19件のテスト / 静的解析 / 本番ビルド

使用技術

FRONTEND	TypeScript / React / Next.js / Tailwind CSS
QUALITY	Vitest / TypeScript / ESLint
WORKFLOW	Node.js / Git / GitHub

提出リンク

自作アプリ	公開後のURLを記入
制作ブログ	自作アプリURLの末尾に /project-story
GitHub	公開後のURLを記入

UtaFit

自分の声で、歌を選ぶ。